PYTHON
90-Day Drill Book

Backend + AI Engineering. Self-test edition.

450 hands-on tasks. Every task has a built-in check, so you never need a teacher.

Basics, Error Handling, Streaming, Async, Databases, FastAPI, LLMs, RAG, Agents

How To Use This Book

Treat Python like a spoken language. You do not learn it by reading, you learn it by speaking every day. Each day has 5 tasks and takes 45 to 60 minutes.

The Daily Routine

- **5 min** - Redo one task from yesterday from memory. No notes.
- **40 min** - Do today's 5 tasks. Write the code yourself in a fresh file.
- **5 min** - Run every check. If a check fails, you are not done.
- **5 min** - Write one line in a notes file: what confused you today.

The Rules

- No copy-paste, no AI writing the answer. Docs and error messages are allowed. Reading docs is a skill.
- Stuck for 15 minutes? Look at the docs, then try again. Only then peek at a hint.
- Every task ends with a *Check* line in green. Prove it with `assert` statements or printed output, not by eye.
- Use a virtual environment per project: `python -m venv .venv`.
- Keep everything in one Git repo: one folder per day (`day01/`, `day02/`...). Commit daily.
- Every 7th day, pick 3 random old tasks and redo them from a blank file. This is the spaced repetition that makes it stick.

Scoring

Mark a task done only when its check passes. Aim for 4 of 5 each day. Below 3 of 5 for two days in a row means repeat the day before moving on.

What You Will Need

Python 3.12 or newer, VS Code, Git, Docker (from Day 66), and a free API key for an LLM provider (from Day 81). Libraries come as you need them: `httpx`, `pytest`, `pydantic`, `sqlalchemy`, `fastapi`, `uvicorn`, `redis`, `numpy`.

Note on checks: where a task says 'verify by hand' or 'you can explain', write the explanation as a comment in your code. If you cannot explain it, you have not learned it yet.

The 90-Day Map

Phase	Days	What You Master
PHASE 1 Core Python	1-20	Strings, collections, functions, files. The vocabulary.
PHASE 2 Error Handling and Resilience	21-30	Exceptions, logging, retries, validation. Code that survives.
PHASE 3 Pythonic Engineering	31-42	OOP, dataclasses, typing, decorators, pytest.
PHASE 4 Data Movement and Streaming	43-58	Generators, chunked I/O, threads, processes, asyncio, sockets, backpressure.
PHASE 5 HTTP, Databases, Validation	59-70	httpx, streaming HTTP, SQL, SQLAlchemy, Postgres, Redis, Pydantic.
PHASE 6 FastAPI Backend	71-80	Routes, DI, errors, auth, streaming responses, WebSockets, shipping.
PHASE 7 AI Engineering	81-90	LLM APIs, structured output, tools, embeddings, RAG, agents, capstone.

Progress Tracker

Tick each day when you pass at least 4 of 5 checks.

[ ] **1**	[ ] **2**	[ ] **3**	[ ] **4**	[ ] **5**	[ ] **6**	[ ] **7**	[ ] **8**	[ ] **9**	[ ] **10**
[ ] **11**	[ ] **12**	[ ] **13**	[ ] **14**	[ ] **15**	[ ] **16**	[ ] **17**	[ ] **18**	[ ] **19**	[ ] **20**
[ ] **21**	[ ] **22**	[ ] **23**	[ ] **24**	[ ] **25**	[ ] **26**	[ ] **27**	[ ] **28**	[ ] **29**	[ ] **30**
[ ] **31**	[ ] **32**	[ ] **33**	[ ] **34**	[ ] **35**	[ ] **36**	[ ] **37**	[ ] **38**	[ ] **39**	[ ] **40**
[ ] **41**	[ ] **42**	[ ] **43**	[ ] **44**	[ ] **45**	[ ] **46**	[ ] **47**	[ ] **48**	[ ] **49**	[ ] **50**
[ ] **51**	[ ] **52**	[ ] **53**	[ ] **54**	[ ] **55**	[ ] **56**	[ ] **57**	[ ] **58**	[ ] **59**	[ ] **60**
[ ] **61**	[ ] **62**	[ ] **63**	[ ] **64**	[ ] **65**	[ ] **66**	[ ] **67**	[ ] **68**	[ ] **69**	[ ] **70**
[ ] **71**	[ ] **72**	[ ] **73**	[ ] **74**	[ ] **75**	[ ] **76**	[ ] **77**	[ ] **78**	[ ] **79**	[ ] **80**
[ ] **81**	[ ] **82**	[ ] **83**	[ ] **84**	[ ] **85**	[ ] **86**	[ ] **87**	[ ] **88**	[ ] **89**	[ ] **90**

PHASE 1

Core Python (Days 1-20)

Strings, collections, functions, files. The vocabulary.

DAY 1 | Variables and Types

Names, types, conversion, truthiness

[ ] 1. Create variables for name (str), age (int), height (float), is_student (bool). Print each with its `type()`.
Check: Output shows str, int, float, bool.

[ ] 2. Swap two variables without a temp variable.
Check: a=1, b=2 becomes a=2, b=1.

[ ] 3. Convert `'42'` to int, `42` to str, `3.9` to int, `'3.14'` to float.
Check: `int(3.9) == 3` (truncates, does not round).

[ ] 4. Predict first, then run: `type(5/2)`, `type(5//2)`, `type(True + 1)`.
Check: float, int, int.

[ ] 5. Predict `bool()` of `0`, `''`, `[]`, `None`, `'0'`, `' '`.
Check: False, False, False, False, True, True.

DAY 2 | Numbers and Math

Arithmetic, divmod, rounding, Decimal

[ ] 1. Average of 5 numbers with a loop, then with `sum()/len()`.
Check: Both give the same value.

[ ] 2. Use `divmod` to convert 3725 seconds to h:m:s.
Check: 1h 2m 5s.

[ ] 3. Run `round(2.675, 2)`. Explain the result, then fix with `Decimal` and `ROUND_HALF_UP`.
Check: Float gives 2.67; Decimal gives 2.68.

[ ] 4. Write `is_prime(n)`. List primes under 30.
Check: 2,3,5,7,11,13,17,19,23,29.

[ ] 5. Compound interest: P=1000, r=5%, 10 years using `**`.
Check: 1628.89 (rounded to 2 dp).

DAY 3 | Strings I

Indexing, slicing, methods

[ ] 1. For `s = 'Python Backend'` print first char, last char, reversed, every 2nd char.
Check: P, d, 'dnekcaB nohtyP', 'Pto akn'.

[ ] 2. Count vowels using `sum()` and a generator expression, no explicit loop.
Check: 'Hello World' gives 3.

[ ] 3. Palindrome check ignoring case, spaces, punctuation.
Check: 'A man, a plan, a canal: Panama' is True.

[ ] 4. Apply `strip`, `upper`, `lower`, `title`, `replace` to `' hello WORLD '`.
Check: title gives ' Hello World '.

[ ] 5. Compare `find` and `index` on a missing substring.
Check: find returns -1; index raises ValueError.

DAY 4 | Strings II

f-strings, split/join, regex basics

[ ] 1. With an f-string and format specs print `Item: Rice | Price: 1,500.50 | Qty: 03`.
Check: Exact match, including the comma and zero-padding.

[ ] 2. Split `'a,b,,c'` on commas and rejoin with `-`.
Check: ['a','b','','c'] then 'a-b--c'.

[ ] 3. Word frequency of a paragraph (lowercase, punctuation stripped).
Check: 'the cat and the hat' gives the: 2.

[ ] 4. Use `re.findall` to extract all emails from a text blob.
Check: Finds a@b.com and x.y@z.org in your sample.

[ ] 5. Mask a card number: `1234567812345678` becomes `************5678`.
Check: Length unchanged, only last 4 visible.

DAY 5 | Conditionals

if/elif/else, match, short-circuit

[ ] 1. FizzBuzz 1 to 30.
Check: 15 is FizzBuzz, 9 is Fizz, 10 is Buzz.

[ ] 2. Grade function: 70+ A, 60-69 B, 50-59 C, 45-49 D, else F; invalid for <0 or >100.
Check: 69 is B, 70 is A, 101 is invalid.

[ ] 3. Leap year function.
Check: 1900 False, 2000 True, 2024 True, 2100 False.

[ ] 4. Map HTTP status codes to messages with a `match` statement (200, 404, 500, default).
Check: 404 gives 'Not Found'.

[ ] 5. Predict `a or 'default'` and `a and a.strip()` for a = None, '', 'hi'.
Check: 'default', 'default', 'hi' for the first; None, '', 'hi' for the second.

DAY 6 | Loops

for, while, enumerate, zip, break

[ ] 1. Print the 7 times table up to 12.
Check: Last line is 7 x 12 = 84.

[ ] 2. Sum the digits of 9875 using a while loop.
Check: 29.

[ ] 3. Use `enumerate(start=1)` and `zip` to print ranked names with scores.
Check: '1. Ada - 90' style lines.

[ ] 4. Find the first number above 100 divisible by both 7 and 11 using while and break.
Check: 154.

[ ] 5. Print a right triangle of stars, height 5, with nested loops.
Check: Row i has i stars.

DAY 7 | Lists

Mutation, copying, common patterns

[ ] 1. Apply append, insert, extend, remove, pop, del. Predict the list after each and assert it.
Check: All asserts pass.

[ ] 2. Remove duplicates but keep original order.
Check: [3,1,3,2,1] gives [3,1,2].

[ ] 3. Rotate a list by k.
Check: [1,2,3,4,5], k=2 gives [4,5,1,2,3].

[ ] 4. `a=[[1],[2]]; b=a.copy(); b[0].append(9)`. Explain why `a` changed, then fix with `copy.deepcopy`.
Check: After the fix, `a` stays [[1],[2]].

[ ] 5. Second largest distinct value without sorting.
Check: [4,9,2,9,7] gives 7.

DAY 8 | Comprehensions

List, dict, set comprehensions

[ ] 1. Squares of the even numbers from 1 to 20 in one comprehension.
Check: [4,16,36,...,400].

[ ] 2. Flatten `[[1,2],[3],[4,5]]` with a comprehension.
Check: [1,2,3,4,5].

[ ] 3. Dict comprehension: word to length for `['apple','fig','banana']`.
Check: {'apple':5,'fig':3,'banana':6}.

[ ] 4. Set comprehension: unique first letters of a word list.
Check: Matches `set(w[0] for w in words)`.

[ ] 5. Rewrite 3 old for-loops as comprehensions, then write one sentence on when NOT to use one.
Check: Your rule is written down.

DAY 9 | Tuples and Sets

Unpacking, immutability, set algebra

[ ] 1. Unpack `a, *rest = (1,2,3,4)`; return 2 values from a function and unpack them.
Check: rest == [2,3,4].

[ ] 2. Show a tuple cannot be reassigned, but a list inside a tuple can change.
Check: TypeError on `t[0]=1`; `t[2].append(5)` works.

[ ] 3. Union, intersection, difference, symmetric difference of two class rosters.
Check: Verify by hand on 4-item sets.

[ ] 4. Find items that appear in exactly one of three lists.
Check: Test with [1,2],[2,3],[3,4] gives {1,4}.

[ ] 5. Use `timeit` to compare `x in list` vs `x in set` on 100k items.
Check: Set lookup is dramatically faster.

DAY 10 | Dictionaries

CRUD, get, merge, sort

[ ] 1. Create, update, delete keys. Use `.get` with default and `.setdefault`.
Check: No KeyError raised anywhere.

[ ] 2. Invert `{'a':1,'b':2}` into `{1:'a',2:'b'}`.
Check: Matches.

[ ] 3. Merge two dicts three ways: `|`, `**`, `update`.
Check: Later dict wins on conflicts.

[ ] 4. Sort a dict by value descending and take the top 3.
Check: Verify with 6 items.

[ ] 5. Nested dict of student to {subject: score}. Compute each student's average.
Check: Verify one by hand.

DAY 11 | Dictionary Patterns

Counter, defaultdict, grouping, indexes

[ ] 1. Use `Counter` on a string; print `most_common(3)`.
Check: 'mississippi' gives s:4, i:4, p:2.

[ ] 2. Group words by length with `defaultdict(list)`.
Check: {3:[...], 5:[...]}.

[ ] 3. Group a list of dicts `{'name','dept'}` by dept.
Check: Each dept lists the right names.

[ ] 4. Build an inverted index: doc_id to words becomes word to set of doc_ids.
Check: Searching a word returns the right docs.

[ ] 5. Two-sum in one pass using a dict.
Check: [2,7,11,15], target 9 gives (0,1).

DAY 12 | Functions I

Args, kwargs, defaults, hints

[ ] 1. Write a function with a default, a keyword-only arg, `*args`, `**kwargs`. Print what each receives.
Check: Names and values match your call.

[ ] 2. Reproduce the mutable default bug `def f(x, acc=[])`, then fix it with `None`.
Check: Two calls do not share state.

[ ] 3. Write `clamp(value, lo, hi)`.
Check: clamp(15,0,10) is 10; clamp(-3,0,10) is 0.

[ ] 4. Return multiple values as a tuple and as a dict; unpack both.
Check: Both unpack correctly.

[ ] 5. Add docstrings and type hints to all four functions. Run `help(fn)`.
Check: Help shows your docstring.

DAY 13 | Functions II

Closures, lambda, recursion, caching

[ ] 1. `make_counter()` returning an increment function using `nonlocal`.
Check: Three calls give 1, 2, 3.

[ ] 2. `sorted` a list of dicts by dept ascending, salary descending using one `key`.
Check: Order verified by eye.

[ ] 3. Recursion: factorial, fibonacci, sum of a nested list.
Check: fib(10) is 55; factorial(5) is 120.

[ ] 4. Memoize fib with `functools.lru_cache`. Time fib(35) before and after.
Check: Cached call under 1 ms.

[ ] 5. `compose(f, g)` returns a function.
Check: compose(str.upper, str.strip)(' hi ') gives 'HI'.

DAY 14 | Mini Project: Text Analyzer

Put days 1-13 together

[ ] 1. CLI that reads a file path from `sys.argv`.
Check: Runs with a path argument.

[ ] 2. Print word count, unique words, top 5 words, longest word, average word length.
Check: Verify on a file you can count by hand.

[ ] 3. Handle empty and missing files without crashing.
Check: Friendly message, no traceback.

[ ] 4. Split logic into functions and use `if __name__ == '__main__':`.
Check: Importing the file runs nothing.

[ ] 5. Write 3 `assert` checks on a known string.
Check: All pass silently.

DAY 15 | Modules and Stdlib

Imports, random, deque, argparse, env

[ ] 1. Create `utils.py` and import from another file.
Check: Works from another file; the `__main__` guard blocks side effects.

[ ] 2. Use `random.seed` then shuffle and sample.
Check: Same seed gives the same result.

[ ] 3. Use `collections.deque` as a queue. Compare with `list.pop(0)` on 100k items.
Check: deque is much faster.

[ ] 4. Build a CLI with `argparse`: one required arg and one optional flag.
Check: `--help` works.

[ ] 5. Read an env var with a default using `os.environ`.
Check: Missing var returns the default, no KeyError.

DAY 16 | itertools and functools

Lazy tools and higher-order functions

[ ] 1. Use `count`, `islice`, `chain`: first 5 evens from an infinite count.
Check: [0,2,4,6,8].

[ ] 2. `itertools.groupby` words by first letter (sort first).
Check: Groups are correct; unsorted input breaks it.

[ ] 3. `combinations` and `permutations` of 3 items; count both.
Check: 3 and 6.

[ ] 4. `functools.reduce` to get a product; then `math.prod`.
Check: Same answer.

[ ] 5. `partial(add_tax, rate=0.075)` to make `pay_vat`.
Check: pay_vat(1000) is 1075.

DAY 17 | Dates and Time

datetime, timedelta, timezones

[ ] 1. Parse `'2025-03-15 14:30'` and print `15 Mar 2025, 2:30 PM`.
Check: Exact format.

[ ] 2. Days between two dates; add 90 days to today.
Check: Check on a calendar.

[ ] 3. Convert UTC now to Africa/Lagos using `zoneinfo`.
Check: Offset is +01:00.

[ ] 4. Is a date on a weekend?
Check: 2025-03-15 is Saturday.

[ ] 5. `age(birthdate)` correct on birthday boundaries.
Check: Birthday tomorrow: age not incremented yet.

DAY 18 | Files I

Read, write, append, encoding

[ ] 1. Write 5 lines to a file; read back via `read()`, `readlines()`, and iteration.
Check: Same content three ways.

[ ] 2. Append without overwriting.
Check: Old lines remain.

[ ] 3. Implement `wc`: lines, words, chars.
Check: Matches the real `wc` output.

[ ] 4. Write emoji text with explicit `encoding='utf-8'` and read it back.
Check: Round-trip is identical.

[ ] 5. Show the file is closed after the `with` block.
Check: `f.closed` is True.

DAY 19 | CSV and JSON

Structured file formats

[ ] 1. Write a list of dicts with `DictWriter`; read it with `DictReader`.
Check: Equal data after the round trip.

[ ] 2. Sum a numeric CSV column (convert to float).
Check: Matches a manual sum.

[ ] 3. `json.dumps` with `indent` and `sort_keys`; dump to a file and reload.
Check: `assert original == loaded`.

[ ] 4. Serialize a dict containing a `datetime` using `default=`.
Check: No TypeError.

[ ] 5. Convert CSV to JSON and JSON to CSV.
Check: Round-trip equals the original.

DAY 20 | pathlib and Review

Filesystem work and a mini report

[ ] 1. With `pathlib` list all `.py` files recursively with sizes.
Check: Matches `find` *or* `dir`.

[ ] 2. Create a nested tree with `mkdir(parents=True, exist_ok=True)`.
Check: Re-running does not error.

[ ] 3. Batch-rename files to `img_001`, `img_002`...
Check: Order preserved.

[ ] 4. Find the 5 largest files in a folder.
Check: Sorted descending by size.

[ ] 5. Review challenge: bank-statement CSV to JSON summary (total in/out, biggest transaction, per-category totals).
Check: Totals reconcile with the CSV.

PHASE 2

Error Handling and Resilience (Days 21-30)

Exceptions, logging, retries, validation. Code that survives.

DAY 21 | try / except Basics

Catching the right errors

[ ] 1. `safe_int(s)` returns None on `ValueError`.
Check: '12' gives 12, 'x' gives None.

[ ] 2. `safe_divide(a, b)` handles `ZeroDivisionError`.
Check: safe_divide(1,0) returns None.

[ ] 3. Trigger and catch KeyError, IndexError, TypeError, AttributeError. Print type name and message.
Check: Four different outputs.

[ ] 4. Show a bare `except:` swallowing Ctrl-C. Replace with `except Exception`.
Check: Ctrl-C now stops the program.

[ ] 5. Write a 3-level nested call that fails. Read the traceback bottom-up.
Check: You can name the failing line and cause.

DAY 22 | else, finally, Multiple Excepts

Full try statement

[ ] 1. Order several `except` clauses from specific to general.
Check: Specific ones catch first.

[ ] 2. Use `try/except/else`: only success code in `else`.
Check: else skipped on error.

[ ] 3. `finally` with a `return` inside `try`.
Check: finally still runs before returning.

[ ] 4. Catch several types with `except (A, B) as e`.
Check: One handler, both types.

[ ] 5. `read_config(path)`: separate messages for FileNotFound, PermissionError, JSONDecodeError.
Check: Three distinct messages.

DAY 23 | Raising and Custom Exceptions

Designing error types

[ ] 1. Raise `ValueError` for a negative age with a helpful message.
Check: Message names the bad value.

[ ] 2. Build `AppError` with children `ValidationError`, `NotFoundError`, `PaymentError`.
Check: `issubclass` checks pass.

[ ] 3. Add extra attributes (`code`, `details`) to a custom exception.
Check: Accessible on the caught error.

[ ] 4. `withdraw(balance, amount)` raises `InsufficientFunds` with `.shortfall`.
Check: withdraw(50,80) gives shortfall 30.

[ ] 5. Catch the base `AppError` and respond differently per subtype.
Check: Each subtype triggers its own branch.

DAY 24 | Exception Chaining

raise from, re-raise, groups

[ ] 1. Wrap a low-level error: `raise NewError(...) from original`.
Check: `e.__cause__ is original`.

[ ] 2. Log and re-raise with bare `raise`.
Check: Original traceback preserved.

[ ] 3. Compare `raise ... from None` with implicit chaining.
Check: Output shows the difference.

[ ] 4. Raise an `ExceptionGroup` of 2 errors and handle each with `except*`.
Check: Both handlers run.

[ ] 5. Layered functions repo, service, api: DB error becomes domain error becomes HTTP-style error, cause kept.
Check: Full chain visible in the traceback.

DAY 25 | Context Managers

with, contextlib, cleanup

[ ] 1. Class-based context manager that times a block.
Check: Prints elapsed time.

[ ] 2. Same thing with `@contextmanager`.
Check: Same behaviour.

[ ] 3. Context manager that changes cwd and always restores it, even on error.
Check: cwd restored after a raised exception.

[ ] 4. Use `contextlib.suppress` and `ExitStack` to open several files.
Check: All files closed afterwards.

[ ] 5. Simulated transaction: commit on success, rollback on exception.
Check: Failing block leaves state unchanged.

DAY 26 | Logging

Replace print with real logs

[ ] 1. Configure logging with timestamp, level, logger name.
Check: Format shows all three.

[ ] 2. Log INFO to console and DEBUG to a file.
Check: Different content in each.

[ ] 3. Use `logger.exception` inside an except block.
Check: Traceback appears in the log.

[ ] 4. Write a custom formatter that emits JSON lines.
Check: Each line parses with `json.loads`.

[ ] 5. Replace all prints in the Day 14 project with logging.
Check: DEBUG hidden at INFO level.

DAY 27 | Validation and Defensive Code

Trust nothing from outside

[ ] 1. `validate_user(dict)` returns a list of all errors (missing email, bad age, ...).
Check: Reports every problem, not only the first.

[ ] 2. Use `assert` for internal invariants only. Test with `python -O`.
Check: Asserts vanish under -O, so never validate user input with them.

[ ] 3. Make a function handle None and empty input gracefully.
Check: No crash on None, '', [].

[ ] 4. Regex-validate email and phone with a 10-row table of inputs and expected results.
Check: All 10 rows match.

[ ] 5. `parse_amount('NGN 1,500.50')` returns `Decimal('1500.50')`; garbage raises ValueError.
Check: Includes at least 5 bad inputs.

DAY 28 | Retry, Backoff, Timeouts

Surviving flaky things

[ ] 1. `retry(fn, attempts=3)` retries on exception.
Check: Gives up after 3 and re-raises.

[ ] 2. Add exponential backoff with jitter.
Check: Logged delays grow roughly 1, 2, 4 seconds.

[ ] 3. Retry only specific exception types.
Check: ValueError is not retried.

[ ] 4. Turn it into `@retry(attempts=3, exceptions=(ConnectionError,))`. Prove it with a function that fails twice then succeeds.
Check: Succeeds on call 3.

[ ] 5. Add a timeout with `concurrent.futures` and `future.result(timeout=...)`.
Check: A slow call raises TimeoutError.

DAY 29 | Debugging

Find bugs fast

[ ] 1. Step through a buggy function with `breakpoint()`. Inspect locals.
Check: You find the bug.

[ ] 2. Write a file with 3 deliberate bugs (off-by-one, mutable default, shadowed builtin) and fix them.
Check: All 3 fixed.

[ ] 3. Capture a traceback as a string with `traceback.format_exc()`.
Check: String contains the failing line.

[ ] 4. Trigger a `DeprecationWarning`; run with `python -X dev`.
Check: Warning displayed.

[ ] 5. Profile a slow function with `cProfile`.
Check: You can name the hotspot.

DAY 30 | Review: Resilient Bulk Processor

Phase 2 capstone

[ ] 1. Recursively scan a folder of CSV files.
Check: All files found.

[ ] 2. Validate each row; collect and log bad rows with reasons.
Check: Bad rows skipped, not fatal.

[ ] 3. Use custom exceptions for file-level and row-level errors.
Check: Different handling for each.

[ ] 4. Log to file with rotation.
Check: Log file exists and is readable.

[ ] 5. Exit codes: 0 all good, 1 partial success, 2 fatal, via `sys.exit`.
Check: `echo $?` *shows the right code.*

PHASE 3

Pythonic Engineering (Days 31-42)

OOP, dataclasses, typing, decorators, pytest.

DAY 31 | Classes Basics

Objects, attributes, methods

[ ] 1. Build `BankAccount` with deposit, withdraw, balance.
Check: Balance updates correctly.

[ ] 2. Add a class attribute counter of accounts created.
Check: Shared across all instances.

[ ] 3. Create two instances and mutate one.
Check: The other is unchanged.

[ ] 4. Add `__repr__` and `__str__`.
Check: `repr` is unambiguous, `str` is friendly.

[ ] 5. Raise errors on invalid operations; cover them with asserts.
Check: Overdraft raises, negative deposit raises.

DAY 32 | Dunder Methods

Make objects behave like built-ins

[ ] 1. `__eq__` and `__hash__` on `Point`.
Check: Two equal points collapse in a set.

[ ] 2. `__lt__` with `functools.total_ordering`; sort a list of objects.
Check: `sorted()` works.

[ ] 3. `__len__`, `__getitem__`, `__iter__` on a `Deck`.
Check: len(), indexing and for-loops all work.

[ ] 4. `__add__` on `Money` with a currency check.
Check: Adding NGN to USD raises.

[ ] 5. `__call__` on an object, and `__enter__/__exit__` on a class.
Check: Both usable naturally.

DAY 33 | Inheritance and Composition

Reuse without a mess

[ ] 1. `Animal` with `Dog` and `Cat`; override a method and call `super()`.
Check: Each speaks differently.

[ ] 2. Build a diamond inheritance case and print `__mro__`.
Check: You can explain the order.

[ ] 3. Refactor inheritance into composition: `Car` has an `Engine`.
Check: Same behaviour, looser coupling.

[ ] 4. Write a `JsonMixin.to_json()` and apply it to two classes.
Check: Both serialize.

[ ] 5. Write 3 sentences: when to inherit, when to compose.
Check: Written down.

DAY 34 | Properties and Class Methods

Controlled attributes

- [ ] **1.** `@property` with a validating setter (age >= 0).
  Check: Negative age raises ValueError.
- [ ] **2.** Read-only computed property `full_name`.
  Check: Setting it raises AttributeError.
- [ ] **3.** `@classmethod from_string('Ada,30')` as alternate constructor.
  Check: Returns a proper instance.
- [ ] **4.** Add a `@staticmethod` helper; explain how it differs from a classmethod.
  Check: Explanation written.
- [ ] **5.** Compare memory of 100k instances with and without `__slots__`.
  Check: Slots version is smaller.

DAY 35 | Dataclasses

Records done right

- [ ] **1.** `@dataclass User` with defaults and `field(default_factory=list)`.
  Check: Two users do not share a list.
- [ ] **2.** Try `frozen=True` and `order=True`.
  Check: Frozen blocks assignment; order enables sorting.
- [ ] **3.** Validate inside `__post_init__`.
  Check: Bad data raises on creation.
- [ ] **4.** `asdict` a nested dataclass and dump it to JSON.
  Check: Loads back equal.
- [ ] **5.** Model the same record as dataclass, TypedDict, namedtuple, dict. Note tradeoffs.
  Check: One line per option.

DAY 36 | Type Hints

Write checkable code

- [ ] **1.** Annotate 5 old functions with `list[int]`, `dict[str,int]`, `X | None`.
  Check: Annotations visible in `help()`.
- [ ] **2.** Define a `TypedDict` for a user payload.
  Check: Used in a function signature.
- [ ] **3.** Write a generic function with `TypeVar`.
  Check: Works for int and str lists.
- [ ] **4.** Use `Callable` and `Literal` in signatures.
  Check: Wrong literal is flagged by the checker.
- [ ] **5.** Run `mypy` or `pyright` on a file and fix every error.
  Check: Zero errors.

DAY 37 | Enums, ABCs, Protocols

Contracts between components

[ ] 1. `Enum` for `OrderStatus`; iterate it and look up by value.
Check: `OrderStatus('paid')` works.

[ ] 2. Use `StrEnum` or `IntEnum` and serialize to JSON.
Check: Dumps as a plain value.

[ ] 3. Abstract base class `PaymentProvider`.
Check: Instantiating it raises TypeError.

[ ] 4. `Protocol` for `Notifier`; two classes satisfy it without inheriting.
Check: Type checker accepts both.

[ ] 5. Two fake payment providers; one function accepts either.
Check: Swap providers with no code change.

DAY 38 | Decorators I

Wrapping behaviour

[ ] 1. `@timer` prints elapsed time.
Check: Prints a number per call.

[ ] 2. `@log_calls` prints args and result.
Check: Shows both.

[ ] 3. Compare `__name__` with and without `functools.wraps`.
Check: With wraps keeps the original name.

[ ] 4. Stack two decorators; predict the execution order.
Check: Your prediction matches.

[ ] 5. Class-based decorator that counts calls.
Check: `fn.calls` increments.

DAY 39 | Decorators II

Decorators with arguments

[ ] 1. `@repeat(3)`.
Check: Function runs 3 times.

[ ] 2. `@retry(attempts, exceptions)` with `wraps`.
Check: Reuses your Day 28 logic.

[ ] 3. `@cache_with_ttl(seconds)`.
Check: Value recomputed after expiry.

[ ] 4. `@require_role('admin')` on functions that take a user dict.
Check: Non-admin raises PermissionError.

[ ] 5. Write a decorator that works on async functions.
Check: Awaited correctly.

DAY 40 | pytest

Tests that protect you

- [ ] **1.** Install pytest; test your `validate` and `parse_amount` functions.
  Check: Green run.
- [ ] **2.** `pytest.raises(match=...)` for error cases.
  Check: Message matched.
- [ ] **3.** `@pytest.mark.parametrize` with 10 cases.
  Check: 10 reported tests.
- [ ] **4.** Test file I/O with the `tmp_path` fixture.
  Check: No leftover files.
- [ ] **5.** Reach 90% coverage with `pytest-cov` on one module.
  Check: Coverage report shows 90%+.

DAY 41 | Fixtures and Mocking

Isolate and simulate

- [ ] **1.** Fixture providing sample data; try function and module scope.
  Check: Scope behaviour verified.
- [ ] **2.** `monkeypatch` an env var and `time.time`.
  Check: Test is deterministic.
- [ ] **3.** `unittest.mock.patch` an HTTP call; assert it was called once with the right args.
  Check: Assertion passes.
- [ ] **4.** Test your retry logic with `Mock(side_effect=[Err, Err, 'ok'])`.
  Check: Returns 'ok' after 3 calls.
- [ ] **5.** Share a fixture across two files via `conftest.py`.
  Check: Both files use it.

DAY 42 | Review: Library System

Phase 3 capstone

- [ ] **1.** Models with dataclasses and Enum (Book, Member, Loan).
  Check: Clean model layer.
- [ ] **2.** Repository interface (ABC) with an in-memory implementation.
  Check: Swappable storage.
- [ ] **3.** Service layer with custom errors (BookUnavailable, LimitExceeded).
  Check: Errors raised in the right places.
- [ ] **4.** Write 15 pytest tests.
  Check: All green.
- [ ] **5.** Type-check with mypy.
  Check: Zero errors.

PHASE 4

Data Movement and Streaming (Days 43-58)

Generators, chunked I/O, threads, processes, asyncio, sockets, backpressure.

DAY 43 | Iterators

How for-loops really work

[ ] 1. Use `iter()` and `next()` on a list; catch `StopIteration`.
Check: Loop ends cleanly.

[ ] 2. Write class `Countdown` with `__iter__` and `__next__`.
Check: Works in a for-loop.

[ ] 3. Show an iterator is exhausted after one pass; explain iterable vs iterator.
Check: Second pass yields nothing.

[ ] 4. Use `iter(callable, sentinel)` to read a file in 4-byte chunks.
Check: Chunks end at EOF.

[ ] 5. Prove `any()` and `all()` short-circuit with a side-effect print.
Check: Print stops early.

DAY 44 | Generators

Lazy, memory-friendly values

[ ] 1. Infinite `fib()` generator; take 10 with `islice`.
Check: 0,1,1,2,3,5,8,13,21,34.

[ ] 2. Compare `sys.getsizeof` of a 1M list comprehension vs generator expression.
Check: Generator is tiny.

[ ] 3. `yield from` to flatten arbitrarily nested lists.
Check: [1,[2,[3]]] gives 1,2,3.

[ ] 4. Use `send()` to build a running-average generator.
Check: Average updates per send.

[ ] 5. Add try/finally to a generator and call `.close()`.
Check: finally block runs.

DAY 45 | Generator Pipelines

Compose streams

[ ] 1. `read_lines(path)` yields lines.
Check: One line at a time.

[ ] 2. `parse(lines)` yields dicts.
Check: Records parse.

[ ] 3. `only_errors(records)` yields status >= 500.
Check: Filter works.

[ ] 4. Chain all three over a generated 1M-line log.
Check: Memory stays flat.

[ ] 5. `batcher(iterable, n)` yields lists of n.
Check: Last partial batch is correct.

DAY 46 | Streaming Files

Handle files bigger than RAM

[ ] 1. Read a big file in 64KB chunks with `while chunk := f.read(...)`.
Check: Total bytes equals file size.

[ ] 2. Copy a big file in chunks with a progress percentage.
Check: Copy hash equals original.

[ ] 3. Implement `tail -f`: follow a file as lines are appended.
Check: New lines appear live.

[ ] 4. SHA-256 of a file in chunks.
Check: Equals the one-shot `hashlib` result.

[ ] 5. Split a big file into parts, then merge them.
Check: Merged hash equals original.

DAY 47 | Memory-Aware Processing

Measure, then optimize

[ ] 1. `tracemalloc`: list vs generator over 5M numbers.
Check: Peak memory differs hugely.

[ ] 2. Stream a generated 1M-row CSV with `csv.reader`.
Check: Constant memory.

[ ] 3. Compare `list` vs `array('i')` for 10M ints.
Check: array is much smaller.

[ ] 4. Top-K with `heapq.nlargest` on a stream.
Check: Matches full sort result.

[ ] 5. Reservoir sampling: k items from a stream of unknown length.
Check: Rough uniformity over many runs.

DAY 48 | Binary Data

Bytes, struct, in-memory files

[ ] 1. `str.encode` and `bytes.decode`, including `errors='replace'`.
Check: Bad bytes become the replacement char.

[ ] 2. `struct.pack` and `unpack` a header (magic, version, length).
Check: Round-trips.

[ ] 3. Use `io.BytesIO` and `io.StringIO` as fake files.
Check: Pass them to a function expecting a file.

[ ] 4. Detect PNG vs PDF vs ZIP from magic bytes.
Check: Three correct detections.

[ ] 5. Base64 an image and decode it back.
Check: Bytes identical.

DAY 49 | Compression and Formats

Move data compactly

[ ] 1. Write and read text lines with `gzip`.
Check: Content intact.

[ ] 2. Create, list and extract one member with `zipfile`.
Check: Only that file extracted.

[ ] 3. Write 100k records as JSON Lines; stream-read them.
Check: One record in memory at a time.

[ ] 4. Compare sizes: JSON, gzip JSON, CSV.
Check: Table of 3 sizes.

[ ] 5. Stream CSV rows into a gzip ndjson file.
Check: Never loads all rows.

DAY 50 | Queues and Producer/Consumer

Thread-safe handoff

[ ] 1. Basic `queue.Queue` put and get.
Check: FIFO order.

[ ] 2. Producer thread and consumer thread, stopped with a sentinel.
Check: Consumer exits cleanly.

[ ] 3. Bounded queue (`maxsize`) to show backpressure.
Check: Fast producer blocks.

[ ] 4. Three consumers on one queue.
Check: Every item processed exactly once.

[ ] 5. Graceful shutdown with `Event` and `join`.
Check: No hung threads.

DAY 51 | Threading

I/O concurrency

[ ] 1. Five 1-second sleeps sequentially vs `ThreadPoolExecutor`.
Check: About 5s vs about 1s.

[ ] 2. Shared counter race condition, then fix with `Lock`.
Check: Final count is exact.

[ ] 3. Compare `as_completed` with `map`.
Check: Different ordering shown.

[ ] 4. Capture a thread's exception via `Future.exception()`.
Check: Error visible in main thread.

[ ] 5. Show CPU-bound work gets no speedup with threads; explain the GIL in 3 lines.
Check: Timing proves it.

DAY 52 | Multiprocessing

CPU concurrency

[ ] 1. `ProcessPoolExecutor` on a CPU-bound prime count.
Check: Faster than threads.

[ ] 2. Pass data between processes with `Queue` or `Pipe`.
Check: Message arrives intact.

[ ] 3. Shared `multiprocessing.Value` with a lock.
Check: Exact total.

[ ] 4. Map-reduce word count across chunks.
Check: Equals single-process result.

[ ] 5. Handle a crashing worker; surface its exception in the parent.
Check: Parent logs the error.

DAY 53 | asyncio Basics

Cooperative concurrency

[ ] 1. `async def`, `await`, `asyncio.run`: hello with sleep.
Check: Runs.

[ ] 2. 5 coroutines sequentially vs `gather`.
Check: About 5x faster with gather.

[ ] 3. `create_task` now, `await` later.
Check: Work overlaps.

[ ] 4. Call a coroutine without awaiting it.
Check: Observe the RuntimeWarning.

[ ] 5. Simulate async fetches with random sleeps; return results in input order.
Check: Order preserved.

DAY 54 | asyncio Control

Timeouts, cancel, groups

[ ] 1. `asyncio.wait_for` with a timeout.
Check: TimeoutError caught.

[ ] 2. Cancel a task; handle `CancelledError` and clean up.
Check: Cleanup runs.

[ ] 3. `gather(return_exceptions=True)` with one failing coroutine.
Check: Others still return.

[ ] 4. `asyncio.TaskGroup`: one failure cancels siblings.
Check: Siblings stop.

[ ] 5. `asyncio.to_thread` for blocking code.
Check: Event loop stays responsive.

DAY 55 | Async Generators

Streams of async data

[ ] 1. `async def` with `yield`: simulate a sensor stream.
Check: Yields over time.

[ ] 2. Consume with `async for` and a timeout.
Check: Timeout handled.

[ ] 3. Async context manager for a fake connection.
Check: Opens and closes.

[ ] 4. Merge two async streams.
Check: Items interleave.

[ ] 5. Pipeline: source, transform, sink as async generators.
Check: Data flows end to end.

DAY 56 | Backpressure and Limits

Control the flow

[ ] 1. `asyncio.Queue(maxsize=N)`; producer awaits when full.
Check: Producer pauses.

[ ] 2. A pool of N workers draining one queue.
Check: All items done.

[ ] 3. `Semaphore` limiting 50 tasks to 5 at a time.
Check: Max concurrent never above 5.

[ ] 4. Rate limiter: at most 10 ops per second.
Check: Timing confirms.

[ ] 5. Graceful shutdown with `queue.join()` and cancelling workers.
Check: No pending-task warnings.

DAY 57 | Sockets

Network streams from scratch

[ ] 1. TCP echo server and client with `socket`.
Check: Echo works.

[ ] 2. Serve many clients with threads.
Check: Two clients at once.

[ ] 3. Rebuild with `asyncio.start_server`.
Check: Same behaviour, no threads.

[ ] 4. Length-prefixed framing: 4-byte length then payload.
Check: Back-to-back messages arrive intact.

[ ] 5. Stream a 10MB file over your socket in chunks.
Check: Received hash equals sent hash.

DAY 58 | Review: Streaming ETL

Phase 4 capstone

- [ ] **1.** Generator producing 1M fake events.
  Check: Never held in memory.
- [ ] **2.** Validate; send bad rows to a dead-letter file.
  Check: Counts add up.
- [ ] **3.** Batch 500 and write to SQLite or gzip.
  Check: Batches written.
- [ ] **4.** Log metrics: rows per second, errors.
  Check: Metrics visible.
- [ ] **5.** Prove memory stays flat with `tracemalloc`.
  Check: Flat peak at 1M rows.

PHASE 5

HTTP, Databases, Validation (Days 59-70)

httpx, streaming HTTP, SQL, SQLAlchemy, Postgres, Redis, Pydantic.

DAY 59 | HTTP Basics

Talking to APIs

[ ] 1. GET a public API with `httpx`; print status, headers, JSON.
Check: 200 and parsed JSON.

[ ] 2. Query params and custom headers; inspect the final URL.
Check: URL encoded properly.

[ ] 3. POST JSON and form data to an echo endpoint.
Check: Server echoes both.

[ ] 4. Use `raise_for_status` and treat 404 vs 500 differently.
Check: Two distinct messages.

[ ] 5. Reuse a `Client` for 20 requests vs 20 separate calls.
Check: Reuse is faster.

DAY 60 | Async HTTP

Many requests safely

[ ] 1. `AsyncClient` fetching 20 URLs with `gather`.
Check: Much faster than sequential.

[ ] 2. Trigger connect vs read timeouts.
Check: Two exception types caught.

[ ] 3. Retry on 429 and 5xx with backoff; honour `Retry-After`.
Check: Delay respects the header.

[ ] 4. Limit concurrency to 5 with a `Semaphore`.
Check: Verified.

[ ] 5. `fetch_json(url)` that never leaks raw exceptions; returns a typed result or error.
Check: Callers need no try/except.

DAY 61 | Streaming HTTP

Large and live responses

[ ] 1. Download a big file via `client.stream`; show progress.
Check: File hash correct.

[ ] 2. `iter_lines` over a line-delimited stream.
Check: Lines arrive as they come.

[ ] 3. Resume a download with a `Range` header.
Check: Final file complete.

[ ] 4. Parse Server-Sent Events by hand (`data:` lines).
Check: Events decoded.

[ ] 5. Connection drops mid-stream: catch, reconnect, continue.
Check: No lost or duplicated items.

DAY 62 | API Patterns

Real-world consumption

[ ] 1. Page-number pagination as a generator.
Check: Yields all items.

[ ] 2. Cursor pagination.
Check: Terminates when cursor is empty.

[ ] 3. Bearer token, API key header, basic auth.
Check: All three work against a mock.

[ ] 4. OAuth2 client-credentials flow against a mock server you write with `http.server`.
Check: Token fetched and reused.

[ ] 5. Disk cache for GET responses with TTL.
Check: Second call hits cache.

DAY 63 | SQLite

Databases without a server

[ ] 1. Create a table; insert and select.
Check: Rows returned.

[ ] 2. Demonstrate SQL injection with an f-string, then fix with parameters.
Check: Injection blocked.

[ ] 3. Use `row_factory` for dict-like rows.
Check: Access by column name.

[ ] 4. `executemany` 100k rows with and without a transaction.
Check: Transaction version is far faster.

[ ] 5. Connection as a context manager; show commit vs rollback.
Check: Rollback leaves no trace.

DAY 64 | SQL Through Python

Joins, indexes, transactions

[ ] 1. Join users and orders; total spent per user.
Check: Verify one by hand.

[ ] 2. `GROUP BY ... HAVING`: users with more than 3 orders.
Check: Correct subset.

[ ] 3. `EXPLAIN QUERY PLAN` before and after an index on 1M rows.
Check: Scan becomes index search.

[ ] 4. Atomic money transfer; force a failure halfway.
Check: Balances unchanged after failure.

[ ] 5. Read a huge table with `fetchmany(1000)`.
Check: Constant memory.

DAY 65 | SQLAlchemy

ORM and repositories

- [ ] **1.** Define User and Post models with a relationship.
  Check: Tables created.
- [ ] **2.** CRUD with a `Session`.
  Check: All four operations work.
- [ ] **3.** Trigger the N+1 problem, then fix with `joinedload`.
  Check: Query count drops.
- [ ] **4.** Alembic migration adding a column.
  Check: Upgrade and downgrade work.
- [ ] **5.** Repository class around the session; test on in-memory SQLite.
  Check: Tests pass without a real DB.

DAY 66 | Postgres and Pooling

Production-grade storage

- [ ] **1.** Run Postgres in Docker; connect with `psycopg`.
  Check: `SELECT 1` works.
- [ ] **2.** Create a schema; load 100k rows with `COPY`.
  Check: Fast bulk load.
- [ ] **3.** Use a connection pool and show reuse.
  Check: Pool stats confirm.
- [ ] **4.** Async SQLAlchemy engine with `asyncpg`.
  Check: Concurrent queries run.
- [ ] **5.** Catch `IntegrityError` on duplicate keys; return a friendly error.
  Check: No raw DB error leaks.

DAY 67 | Redis

Cache, limits, messaging

- [ ] **1.** SET and GET with expiry.
  Check: Key vanishes after TTL.
- [ ] **2.** Cache-aside pattern around a slow function.
  Check: Second call instant.
- [ ] **3.** Rate limiter with `INCR` and `EXPIRE`.
  Check: 11th call in a window blocked.
- [ ] **4.** Pub/Sub: publisher and subscriber scripts.
  Check: Messages delivered live.
- [ ] **5.** Redis Streams with a consumer group.
  Check: Each message processed once.

DAY 68 | Pydantic

Validated data models

- [ ] **1.** `BaseModel` for User; build from a dict.
  Check: Fields typed.
- [ ] **2.** Constraints: `min_length`, `ge`, regex pattern, `EmailStr`.
  Check: Bad input rejected.
- [ ] **3.** Catch `ValidationError`; print `errors()`.
  Check: Structured error list.
- [ ] **4.** Nested models and a list of models from JSON.
  Check: Parses deep structure.
- [ ] **5.** `model_dump_json` and `model_validate_json` round trip.
  Check: Equal models.

DAY 69 | Pydantic II

Advanced validation

- [ ] **1.** `field_validator` and `model_validator`.
  Check: Cross-field rule enforced.
- [ ] **2.** Aliases: camelCase JSON to snake_case fields.
  Check: Both directions work.
- [ ] **3.** `pydantic-settings` loading from `.env`.
  Check: Env override beats file.
- [ ] **4.** Discriminated union (`type: card | bank`).
  Check: Right model chosen.
- [ ] **5.** Validate a 100k-record ndjson stream; count errors by field.
  Check: Summary table printed.

DAY 70 | Review: Ingestion Service

Phase 5 capstone

- [ ] **1.** Pull a paginated mock API with retries and timeouts.
  Check: Survives injected failures.
- [ ] **2.** Validate with Pydantic; reject and log bad records.
  Check: Reject count reported.
- [ ] **3.** Upsert into SQLite or Postgres.
  Check: Re-running creates no duplicates.
- [ ] **4.** Cache responses; skip unchanged pages.
  Check: Second run is fast.
- [ ] **5.** Log metrics and exit with proper codes.
  Check: Report at end of run.

PHASE 6

FastAPI Backend (Days 71-80)

Routes, DI, errors, auth, streaming responses, WebSockets, shipping.

DAY 71 | FastAPI First Steps

Routes, params, docs

[ ] 1. Hello app with `uvicorn --reload`; open `/docs`.
Check: Swagger UI loads.

[ ] 2. Path and query params with types and defaults; send a bad type.
Check: 422 with a clear message.

[ ] 3. In-memory CRUD for `/items` (list, get, create, update, delete).
Check: All five endpoints work.

[ ] 4. Correct status codes: 201 on create, 204 on delete, 404 when missing.
Check: Verified with curl.

[ ] 5. Add tags, summaries, descriptions; inspect `/openapi.json`.
Check: Docs read well.

DAY 72 | Request and Response Models

Schemas that protect you

[ ] 1. Accept a Pydantic body.
Check: Invalid body gives 422.

[ ] 2. `response_model` that hides the password field.
Check: Password never returned.

[ ] 3. Separate Create, Read, Update schemas; implement partial PATCH.
Check: Only sent fields change.

[ ] 4. Filtering plus `limit` and `offset` with bounded validation.
Check: limit=10000 rejected.

[ ] 5. Add a custom response header.
Check: Visible in curl -i.

DAY 73 | Dependency Injection

Reusable building blocks

[ ] 1. Pagination params as a dependency.
Check: Reused by 2 routes.

[ ] 2. DB session dependency with `yield` cleanup.
Check: Session closes after each request.

[ ] 3. Dependency chain: `get_current_user` depends on a token.
Check: Chain resolves.

[ ] 4. Override a dependency in a test.
Check: Test uses the fake.

[ ] 5. Split code into `APIRouter` modules with prefix and shared dependencies.
Check: Clean file layout.

DAY 74 | FastAPI Error Handling

Predictable failures

[ ] 1. `HTTPException` with detail and headers.
Check: Right status and body.

[ ] 2. Custom exception plus `@app.exception_handler` returning `{code, message}`.
Check: Consistent error shape.

[ ] 3. Override the `RequestValidationError` handler for cleaner output.
Check: Friendly field errors.

[ ] 4. Global handler for unhandled `Exception`: return 500 without leaking the traceback; log with a request id.
Check: Client sees no internals.

[ ] 5. Map your domain errors (Day 23) to status codes; test every path.
Check: NotFound 404, Validation 422, Conflict 409.

DAY 75 | Authentication

Passwords and JWT

[ ] 1. Hash passwords with argon2 or bcrypt; verify them.
Check: Plain text never stored.

[ ] 2. `POST /register` and `POST /login` returning a JWT.
Check: Token returned.

[ ] 3. Decode JWT; separate expired from invalid tokens.
Check: Distinct 401 messages.

[ ] 4. Protect a route with `OAuth2PasswordBearer`.
Check: No token means 401.

[ ] 5. Role-based access (admin only) and refresh tokens.
Check: Non-admin gets 403.

DAY 76 | FastAPI with a Database

Real persistence

[ ] 1. Async SQLAlchemy session dependency.
Check: Used in all routes.

[ ] 2. Models and CRUD for posts owned by users.
Check: Owner stored.

[ ] 3. Only the owner can edit a post.
Check: Others get 403.

[ ] 4. Duplicate data gives 409 via `IntegrityError` handling.
Check: Clear message.

[ ] 5. Alembic migrations plus a seed script.
Check: Fresh DB builds with 2 commands.

DAY 77 | Background Tasks and Middleware

Cross-cutting concerns

[ ] 1. `BackgroundTasks` sending a fake email after the response.
Check: Response returns first.

[ ] 2. Middleware adding `X-Request-ID` and logging duration.
Check: Header on every response.

[ ] 3. CORS for one specific origin.
Check: Other origin blocked.

[ ] 4. Redis-backed rate limit middleware (Day 67).
Check: 429 after the limit.

[ ] 5. Lifespan: open DB pool on startup, close on shutdown.
Check: No leaked connections.

DAY 78 | Streaming in FastAPI

Move data without loading it

[ ] 1. `StreamingResponse` of a slow generator.
Check: Chunks arrive over time in `curl -N`.

[ ] 2. Stream a large file download in chunks.
Check: Memory stays flat.

[ ] 3. Server-Sent Events endpoint (`text/event-stream`) pushing an event every second.
Check: Browser or curl shows live events.

[ ] 4. Streaming upload via `UploadFile`; write in chunks; return 413 over a size limit.
Check: Large file rejected early.

[ ] 5. Stream a 1M-row table as ndjson.
Check: Constant memory, starts immediately.

DAY 79 | WebSockets

Two-way live data

[ ] 1. Echo WebSocket.
Check: Messages bounce back.

[ ] 2. Connection manager that broadcasts to all clients.
Check: 2 browsers see each other.

[ ] 3. Authenticate a WebSocket with a token.
Check: Bad token refused.

[ ] 4. Handle `WebSocketDisconnect` cleanly.
Check: No error spam, client removed.

[ ] 5. Chat rooms with join, leave, room-only broadcast.
Check: Messages stay in their room.

DAY 80 | Test and Ship

Phase 6 capstone

[ ] 1. Tests for every endpoint with `TestClient` or `AsyncClient`.
Check: All green.

[ ] 2. Test the full auth flow and every error response.
Check: Error paths covered.

[ ] 3. Dockerfile for the app.
Check: Container serves requests.

[ ] 4. `docker-compose` with app, Postgres, Redis.
Check: One command starts all.

[ ] 5. `/health` checks DB and Redis; returns 503 if either is down.
Check: Stop Redis and confirm 503.

PHASE 7

AI Engineering (Days 81-90)

LLM APIs, structured output, tools, embeddings, RAG, agents, capstone.

DAY 81 | LLM API Basics

Your first AI calls

[ ] 1. Install an LLM SDK, make a first call, print text and token usage.
Check: Response plus usage numbers.

[ ] 2. System prompt plus multi-turn history in a list.
Check: Model remembers earlier turns.

[ ] 3. Vary `max_tokens` and temperature; read `stop_reason`.
Check: You can trigger a truncation.

[ ] 4. Handle auth errors, 429 rate limits, overload, timeouts, each differently with retries.
Check: Simulated failures handled.

[ ] 5. Log token usage and estimated cost per request.
Check: Running total printed.

DAY 82 | Prompts and Structured Output

Reliable data from models

[ ] 1. Prompt template function with variables.
Check: Reusable across inputs.

[ ] 2. Request JSON only and parse with `json.loads`; handle failure.
Check: Bad output does not crash you.

[ ] 3. Validate with Pydantic; on failure re-ask with the error (max 2 tries).
Check: Repair loop fixes broken output.

[ ] 4. Extract name, date, amount from 10 messy texts.
Check: 10 structured records.

[ ] 5. Tiny eval: expected vs actual, accuracy percent.
Check: A number you can improve.

DAY 83 | Streaming LLM Output

Token-by-token delivery

[ ] 1. Stream tokens to the terminal.
Check: Text appears progressively.

[ ] 2. Accumulate full text and final usage from the stream.
Check: Totals match non-streaming.

[ ] 3. Expose it as a FastAPI SSE endpoint.
Check: Browser receives live tokens.

[ ] 4. Client disconnects mid-stream: stop and clean up.
Check: No orphan generation.

[ ] 5. Provider error mid-stream: emit an error event and close cleanly.
Check: Client gets a clear final event.

DAY 84 | Tool Use

Let the model call your code

[ ] 1. Define a tool schema (`get_weather`).
Check: Model requests it.

[ ] 2. Detect the tool call, run your function, return the result.
Check: Model uses the result in its answer.

[ ] 3. Loop until the model stops asking for tools.
Check: Multi-step tasks complete.

[ ] 4. Add a calculator and a DB lookup; let the model choose.
Check: Picks the right tool.

[ ] 5. Return tool errors as error results; cap max iterations.
Check: Model recovers; no infinite loop.

DAY 85 | Embeddings

Meaning as numbers

[ ] 1. Embed 20 sentences.
Check: 20 vectors of equal length.

[ ] 2. Cosine similarity from scratch with numpy.
Check: Matches a library result.

[ ] 3. Find the 3 nearest sentences to a query.
Check: Results make sense.

[ ] 4. Normalize vectors; show dot product equals cosine.
Check: Values equal.

[ ] 5. Batch embedding with rate limiting and a cache keyed by text hash.
Check: Repeat run makes zero API calls.

DAY 86 | Vector Search

Find by meaning at scale

[ ] 1. Brute-force search over 10k vectors; time it.
Check: Baseline latency recorded.

[ ] 2. Repeat with FAISS or Chroma.
Check: Faster, same top results.

[ ] 3. Store and query vectors in Postgres with pgvector.
Check: SQL similarity query works.

[ ] 4. Add metadata filtering to search.
Check: Filtered results only.

[ ] 5. Measure recall@5 on labeled queries.
Check: A recall percentage.

DAY 87 | Chunking and RAG

Answer from your documents

- [ ] **1.** Chunk text by size with overlap.
  Check: Overlap visible.
- [ ] **2.** Chunk by headings or sentences.
  Check: Chunks stay coherent.
- [ ] **3.** Ingest pipeline: files, chunks, embeddings, store; idempotent.
  Check: Re-run adds no duplicates.
- [ ] **4.** Retrieve top-k and build a context prompt with source ids.
  Check: Prompt cites ids.
- [ ] **5.** Answer with citations; say 'I don't know' below a relevance threshold.
  Check: Off-topic question gets a refusal.

DAY 88 | RAG Service

Ship it as an API

- [ ] **1.** `POST /ingest` for document upload.
  Check: Doc searchable after upload.
- [ ] **2.** `POST /ask` returning answer plus sources.
  Check: Sources included.
- [ ] **3.** Streaming `/ask` over SSE.
  Check: Tokens stream.
- [ ] **4.** Conversation memory trimmed to a token budget.
  Check: Long chats do not overflow.
- [ ] **5.** Per-user auth and rate limits; log latency and tokens.
  Check: Metrics per request.

DAY 89 | Agents

Loops that act

- [ ] **1.** Minimal agent loop: think, tool, observe, answer, with max steps.
  Check: Stops by itself.
- [ ] **2.** Tools: allowlisted web fetch, sandboxed file read, read-only SQL.
  Check: Each tool works.
- [ ] **3.** Guardrails: validate tool args, block path traversal, confirm destructive actions.
  Check: `../../etc/passwd` is refused.
- [ ] **4.** Per-tool timeouts and retries; circuit breaker after N failures.
  Check: Failing tool gets disabled.
- [ ] **5.** Log every step as JSON; replay a run from the log.
  Check: Replay reproduces the trace.

DAY 90 | Capstone: AI Backend Service

Everything together

- [ ] **1.** FastAPI service with JWT auth, Postgres, Redis.
  Check: Boots with docker-compose.
- [ ] **2.** RAG over your own notes plus a tool-using agent endpoint.
  Check: Cited answers; tools work.
- [ ] **3.** SSE streaming for answers; WebSocket optional.
  Check: Live output.
- [ ] **4.** Full error handling: retries, timeouts, typed errors, health checks, structured logs.
  Check: Kill a dependency and it degrades gracefully.
- [ ] **5.** Tests, README with architecture diagram, deployed to a real host.
  Check: Public URL works.

Appendix: The Action Bank

The most-used Python 'words'. Once a week, pick 10 at random and write a one-line use of each from memory. If you hesitate, drill it.

Strings

split, join, strip, lstrip, rstrip, replace, find, index, startswith, endswith, upper, lower, title, capitalize, isdigit, isalpha, zfill, center, ljust, rjust, format, f-string specs, encode, decode, partition, splitlines, count, translate

Lists

append, extend, insert, remove, pop, clear, index, count, sort, sorted, reverse, reversed, copy, slicing, unpacking, enumerate, zip, min, max, sum, any, all, len, in, list comprehension

Dicts and Sets

get, setdefault, update, pop, popitem, keys, values, items, del, in, dict comprehension, merge with |, Counter, defaultdict, OrderedDict, add, discard, union, intersection, difference, issubset

Functions

def, return, default args, *args, **kwargs, keyword-only, lambda, closures, nonlocal, global, recursion, lru_cache, partial, reduce, map, filter, sorted(key=), docstrings, type hints, decorators, generators, yield, yield from

Files and OS

open, read, readline, readlines, write, with, pathlib.Path, exists, mkdir, glob, rglob, read_text, write_text, rename, unlink, os.environ, os.getcwd, shutil.copy, shutil.rmtree, tempfile, sys.argv, argparse

Data Formats

json.load, json.dump, json.loads, json.dumps, csv.reader, csv.DictReader, csv.DictWriter, gzip.open, zipfile.ZipFile, pickle (and its dangers), base64, hashlib, uuid, struct.pack, io.BytesIO, io.StringIO, tomllib

Errors

try, except, else, finally, raise, raise from, custom exceptions, ExceptionGroup, assert, with, contextmanager, suppress, ExitStack, traceback, logging.exception, warnings.warn, sys.exit

Time and Math

datetime.now, strptime, strftime, timedelta, zoneinfo, time.sleep, perf_counter, math.floor, ceil, sqrt, isclose, Decimal, Fraction, random.choice, shuffle, sample, randint, seed, statistics.mean, median

OOP

class, __init__, self, __repr__, __str__, __eq__, __hash__, __lt__, __len__, __iter__, __getitem__, __enter__, __exit__, __call__, @property, @classmethod, @staticmethod, @dataclass, ABC, Protocol, Enum, __slots__, super, isinstance, issubclass

Streaming and Concurrency

iter, next, generator expressions, itertools.islice, chain, groupby, tee, queue.Queue, threading.Thread, Lock, Event, ThreadPoolExecutor, ProcessPoolExecutor, asyncio.run, gather, create_task, TaskGroup, wait_for, Semaphore, Queue, to_thread, async for, async with

Networking and HTTP

httpx.get, post, Client, AsyncClient, stream, iter_lines, raise_for_status, timeout, headers, params, json, socket.bind, listen, accept, recv, sendall, asyncio.start_server, urllib.parse, http.server

Databases

sqlite3.connect, execute, executemany, fetchone, fetchall, fetchmany, commit, rollback, parameterized queries, SQLAlchemy Session, select, join, relationship, Alembic, psycopg, asyncpg, redis get, set, incr, expire, publish, xadd

Web and Validation

FastAPI route, Depends, HTTPException, APIRouter, BackgroundTasks, StreamingResponse, WebSocket, middleware, lifespan, BaseModel, Field, field_validator, model_dump, model_validate, pydantic-settings, JWT encode and decode, password hashing

Testing and Tooling

pytest, assert, raises, parametrize, fixture, tmp_path, monkeypatch, mock.patch, coverage, mypy, ruff, black, venv, pip, pip freeze, requirements.txt, pyproject.toml, git commit, git branch, Docker build, docker compose up

AI Engineering

LLM messages API, system prompt, max_tokens, temperature, streaming, tool schema, tool_result, structured JSON output, Pydantic repair loop, embeddings, numpy dot, cosine similarity, chunking with overlap, FAISS or pgvector, top-k retrieval, citations, token budgeting, agent loop, guardrails

Final Word

Ninety days of honest daily practice puts you ahead of most people who have 'learned' Python for years. Skip nothing, cheat nowhere, and ship the Day 90 capstone. That project is your proof.